PROJET Cloud Healthcare Unit

Livrable 2

Référentiel de Données et Architecture

CESI
ÉCOLE D'INGÉNIEURS

FISA INFO A4 - 2026/2027

Lisa ACHOUR - Enzo CADIÈRE - Romain TOUZÉ- Thaïs VIANES

Sommaire

Sommaire **2**
Table des figures **3**
Remis en contexte **4**
Description technique **5**
a. explication couches Bronze/Silver/Gold 5
b. technologies retenues 5
c. Flux technologique 6
Plan de gouvernance des données **8**
a. Réglementaire 8
b. RGPD 8
c. HDS 8
d. OIDC 9
Spécifications fonctionnelles **11**
a. Sources 11
b. Nettoyage 11
c. Transformation 11
d. Gestion des erreurs 12
e. Planification des tâches 12
Plan conceptuel de données **13**
a. Tables de faits 14
b. Table de Dimensions 14
Script d'ingestion de données **15**
Scripts de création des données **17**
Scripts de chargement des données **22**
Partitionnement et indexation **24**
a. Stratégie par table 24
b. Scripts 26
Rapport de Qualité des données **27**
a. Complétude 27
b. Cohérence référentielle et codes FINESS 27
c. Doublons 28
d. Contrôles automatiques 29
Logs et traces **30**
a. Logs, métriques et traces 30
b. Centralisation des logs 30
c. Métriques et supervision 30
d. Traçabilité des accès aux données 31
e. Traçabilité des exécutions 31
f. Données de santé dans les logs 31
Benchmark et performances **32**
a. Configurations comparées 32
b. Protocole 32
c. Requêtes de benchmark 32
d. Résultats avant et après optimisation 34
e. Analyse et interprétation des résultats 35
f. Limites de la mesure 36

Conclusion **38**
Annexes **39**
Annexe 1 : Glossaire **39**
Annexe 2 : Requêtes de création des tables et de chargement des données 40

Table des figures

Figure 1 - Flux de données dans l'ETL **6**
Figure 2 - Plan conceptuel de Données **13**
Figure 3 - Lecture des fichiers CSV depuis S3. **15**
Figure 4 - Exclusion des colonnes sensibles. **15**
Script 1 - Création Table dim_patient **17**
Script 2 - Création de la table dim_diagnostic **17**
Script 3 - Création de la table dim_professionnel **17**
Script 4 - Création de la table dim_etablissement **18**
Script 5 - Création de la table dim_date **18**
Script 6 - Création de la table dim_localisation **18**
Script 7 - Création de la table dim_type_satisfaction **19**
Script 8 - Création de la table fact_consultation **19**
Script 9 - Création de la table fact_hospitalisation **20**
Script 10 - Création de la table fact_deces **20**
Script 11 - Création de la table fact_satisfaction **20**
Script 12 - Création de la table agg_deces **21**
Script 13 - Création de la table agg_satisfaction_region **21**
Script 14 - Chargement de la table agg_deces **22**
Script 15 - Chargement de la table dim_patient **22**
Script 16 - Chargement de la table fact_deces **23**
Figure 5 - Double élagage : partitions puis granules (schéma illustratif) **24**
Tableau 1 - Partitionnement et clé de tri proposés par table **25**
Tableau 2 - Volumétrie des tables de l'entrepôt **25**
Script 17 - Partition et clé de tri des tables de faits (A2) (extrait, lignes 29 à 41) **26**
Script 18 - Index de saut (A3) (extrait, lignes 21 à 36) **26**
Figure 6 - Taux de complétude par table, avant et après nettoyage **27**
Tableau 3 - Règles de cohérence référentielle et résultats **28**
Tableau 4 - Doublons par entité et traitement **29**
Tableau 5 - Requêtes de benchmark et levier d'optimisation attendu **33**
Script 19 - SQL Requête Q5 **33**
Tableau 6 - Temps à chaud (ms) **34**
Tableau 7 - Coût des optimisations : taille sur disque et durée de chargement **34**
Tableau 8 - Hypothèses de départ et constats mesurés **35**

Remis en contexte

Le groupe hospitalier CHU mène une transformation numérique de son système d'information. Il souhaite exploiter les grandes quantités de données produites par ses outils de gestion des soins pour mieux piloter son activité et améliorer la qualité des soins. Dans le cadre du projet *Cloud Healthcare Unit*, notre équipe doit concevoir un entrepôt de données décisionnel. Celui-ci rassemblera des données d'origines variées : l'activité médicale des patients, les informations sur les établissements, la satisfaction des usagers et les données nationales de mortalité. L'objectif est d'offrir une plateforme d'analyse unique aux praticiens comme à la direction des établissements.

Les analyses attendues portent principalement sur les taux de consultation et d'hospitalisation, que l'on pourra observer selon l'établissement, le diagnostic, le professionnel, le sexe, l'âge ou la période. La mortalité et la satisfaction par région viennent en complément.

Comme il s'agit de données de santé à caractère personnel, la solution doit respecter le Règlement Général sur la Protection des Données (RGPD), la certification Hébergeur de Données de Santé (HDS) et les recommandations de la Commission Nationale de l'Informatique et des libertés (CNIL), ce qui impose une gouvernance des données rigoureuse. Sur le plan technique, l'architecture suit le modèle Lakehouse en trois couches, avec une modélisation en étoile. Enfin, la solution doit offrir un bon rapport coût-efficacité, garantir la sécurité des données et pouvoir monter en charge.

Description technique

a. explication couches Bronze/Silver/Gold

1. Couche Bronze : stockage des données brutes

La couche Bronze constitue la première étape du traitement des données. Elle permet de collecter et de stocker les données provenant des sources d'origine, sans transformations importantes.

Les données sont conservées dans leur format initial (CSV, Excel, JSON), sans corriger les éventuelles erreurs, doublons ou valeurs manquantes. Certaines données personnelles non nécessaires peuvent toutefois être supprimées dès l'importation.

Des métadonnées peuvent également être ajoutées pour assurer la traçabilité des données.

À la fin de cette étape, les données brutes sont stockées et prêtes à être traitées.

2. Couche Silver : nettoyage et fiabilisation des données

La couche Silver permet de transformer les données brutes en données propres, cohérentes et structurées.

Cette étape consiste à corriger les formats, harmoniser les catégories, traiter les doublons et identifier les valeurs manquantes ou les anomalies.

Des mesures de protection des données personnelles peuvent également être appliquées, comme le chiffrement ou la pseudo-anonymisation.

À la fin de cette étape, les données sont fiabilisées et suffisamment propres pour être utilisées dans les analyses.

3. Couche Gold : récupération des données et analyse

La couche Gold constitue la dernière étape de l'architecture. Elle permet de transformer les données nettoyées en informations directement exploitables pour répondre aux besoins du projet.

Elle contient des tables analytiques et des indicateurs calculés à partir de regroupements, de moyennes, de pourcentages ou de comparaisons.

À la fin de cette étape, les données sont prêtes à être utilisées pour réaliser des graphiques, construire des tableaux de bord et interpréter les résultats.

b. technologies retenues

Pour rappel, l’architecture retenue au livrable 1 repose sur une stack entièrement open source, organisée en Lakehouse (Bronze / Silver / Gold) sur un stockage objet S3 au

format Parquet. Airflow orchestre la chaîne. Spark ingère les sources et applique la pseudo-anonymisation, puis dbt, exécuté par Trino, transforme les données jusqu'à la couche Gold. Cette couche est copiée dans ClickHouse pour être restituée sous Superset. Elasticsearch et Grafana assurent la centralisation des journaux et la supervision. Ces choix répondent aux exigences de coût, d'hébergement HDS, de scalabilité et de réversibilité du sujet.

c. Flux technologique

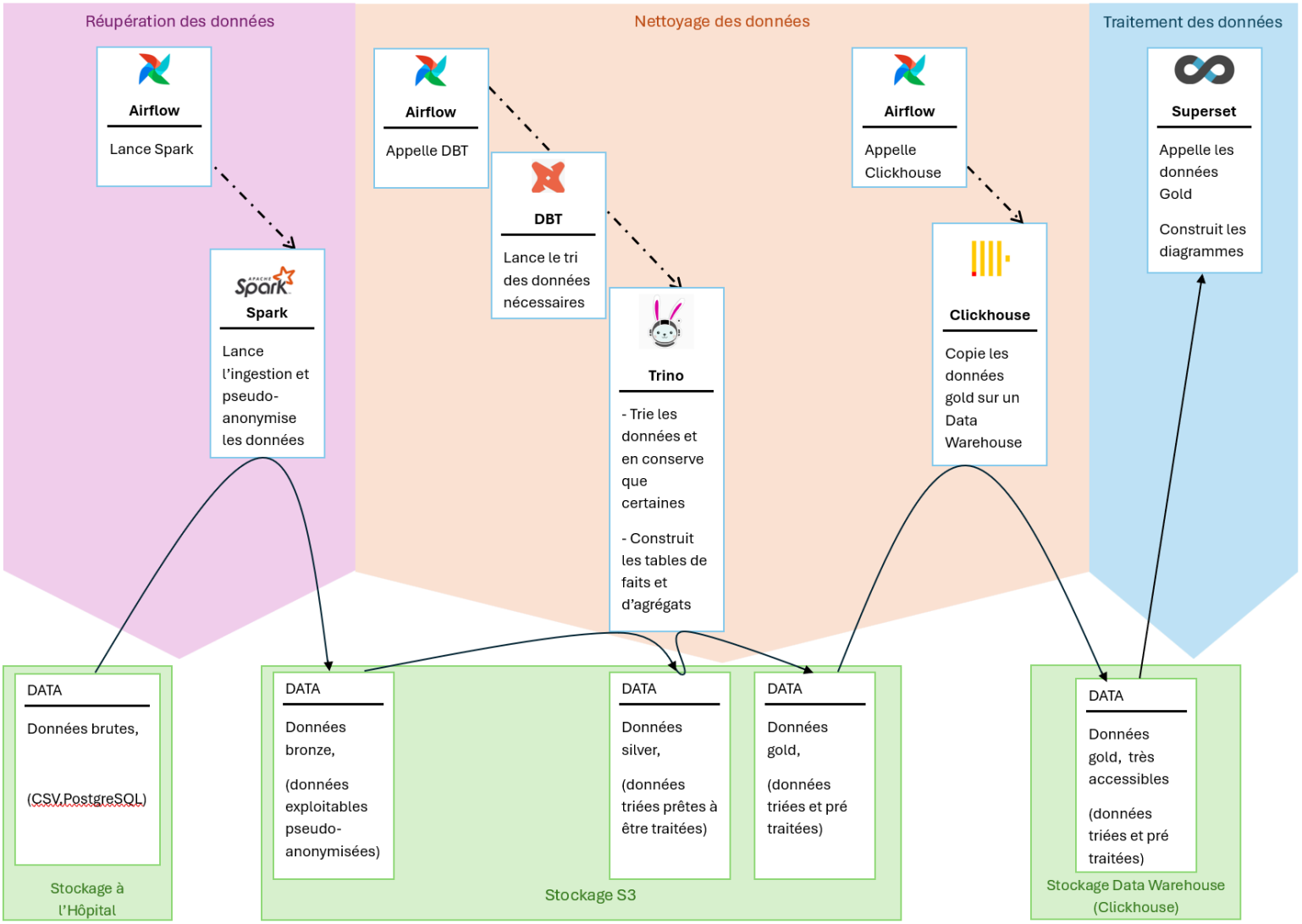


Figure 1 - Flux de données dans l'ETL

Le transit des données s'effectue en plusieurs étapes, des sources hospitalières jusqu'à leur visualisation dans Superset.

Dans un premier temps, Airflow déclenche Spark, qui récupère les données CSV et PostgreSQL, applique les premiers traitements de protection, comme la pseudo-anonymisation, puis les stocke dans la couche Bronze sur S3.

Ensuite, Airflow lance dbt, qui utilise Trino pour nettoyer, normaliser et structurer les données dans la couche Silver, avant de créer les tables analytiques et les agrégats dans la couche Gold.

Une fois les traitements terminés, les données Gold sont transférées vers ClickHouse, qui sert d'entrepôt analytique.

Enfin, Superset interroge ClickHouse pour générer des graphiques et des tableaux de bord permettant de visualiser les indicateurs statistiques issus des données hospitalières.

Plan de gouvernance des données

a. Réglementaire

Pour rappel, notre plan de gouvernance s'appuie sur les textes de l'ANS (dont les guides de la PGSSI-S), de l'Assurance Maladie et du Code de la santé publique. Il retient quatre mesures.

La première est de minimiser les données, en ne collectant que les champs nécessaires aux calculs.

La deuxième est de pseudo-anonymiser les données dès l'ingestion. La troisième est de sécuriser l'environnement d'analyse, qui doit être dédié, à accès restreint et journalisé. La dernière est d'anonymiser les résultats avant diffusion, pour éviter toute réidentification par les petits effectifs ou par le croisement de variables rares.

b. RGPD

1. Finalité et minimisation

Conformément au RGPD et aux recommandations de la CNIL, les données doivent être utilisées uniquement pour produire des statistiques hospitalières. Seules les informations nécessaires doivent être conservées, en supprimant les données directement identifiantes lorsqu'elles ne sont pas indispensables. Le traitement des données de santé doit également reposer sur une base juridique adaptée.

2. Pseudo-anonymisation et sécurité

Les identifiants personnels doivent être remplacés par des identifiants techniques lorsque le suivi individuel est nécessaire. L'accès aux données doit être limité aux personnes autorisées, avec une gestion des droits et une journalisation des opérations. Les données doivent également être protégées lors de leur stockage et de leur transfert.

3. Conservation et traçabilité

Une durée de conservation adaptée doit être définie selon la finalité du traitement. Les opérations doivent être documentées dans un registre et les personnes concernées doivent être informées. Enfin, une AIPD doit être réalisée lorsque le traitement présente un risque élevé pour les droits et libertés des personnes.

c. HDS

Pour rappel, la certification HDS (article L. 1111-8 du Code de la santé publique) s'applique à l'hébergeur des données de santé, et non directement à notre pipeline. Nous

devons donc choisir un prestataire certifié sur les activités que nous utilisons : infrastructure, plateforme, administration ou sauvegarde.

Le référentiel HDS de l'ANS, qui s'appuie sur ISO 27001, est complété par la PGSSI-S. Dans notre architecture, ces exigences se traduisent par le chiffrement des données, la gestion des habilitations, la journalisation, ainsi que des sauvegardes protégées avec des procédures de restauration et de suppression.

d. OIDC

1. Principe

OpenID Connect (OIDC) est un protocole d'authentification basé sur OAuth 2.0. Il permet de centraliser l'identification des utilisateurs grâce à un fournisseur d'identité. L'utilisateur se connecte une seule fois pour accéder aux différents outils (SSO), sans avoir à stocker plusieurs mots de passe.

2. Fournisseur d'identité retenu

Keycloak a été choisi comme fournisseur d'identité. Cette solution open source permet de gérer les comptes, les groupes, les mots de passe et l'authentification multifacteur (MFA). Hébergé sur une infrastructure certifiée HDS, il permet de conserver la maîtrise des identités et des journaux de connexion.

3. Intégration aux outils

Superset, Airflow et Grafana utilisent directement OIDC pour déléguer leur authentification à Keycloak. Les autres interfaces sont protégées par un proxy d'authentification qui vérifie les sessions avant d'autoriser l'accès.

4. Gestion des rôles et des habilitations

Les droits sont centralisés dans Keycloak sous forme de groupes, puis transmis aux différents outils :

- Équipe technique : administration des outils techniques.
- Direction : accès en lecture aux tableaux de bord d'activité.
- Praticiens : accès aux tableaux de bord concernant leur activité médicale.

Les droits sont mis à jour automatiquement lors des connexions. La désactivation d'un compte ou un changement de groupe s'applique ainsi à l'ensemble des outils.

5. Sécurité et traçabilité

Keycloak conserve les journaux de connexion, les échecs d'authentification et les modifications de droits. La MFA, la limitation de la durée des sessions et la protection contre les tentatives de connexion réduisent les risques d'usurpation de compte. Les échanges sont également chiffrés pour protéger les données.

6. Perspectives

Keycloak pourra être relié à Pro Santé Connect (PSC), le fournisseur d'identité de l'ANS. Les professionnels de santé pourront ainsi se connecter avec leur carte professionnelle (CPS, CPE ou CPF), sans avoir besoin de créer un compte supplémentaire.

Spécifications fonctionnelles

a. Sources

Les données fournies par le groupe HCG se répartissent entre des fichiers CSV et une base PostgreSQL. Les fichiers CSV portent sur les professionnels et les établissements de santé (avec leur FINESS), les décès, les hospitalisations (séjours et diagnostics) et la satisfaction (scores par établissement et par année, accompagnés d'un lexique). La base PostgreSQL sante regroupe les actes médico-administratifs. Nous en exploitons principalement les tables diagnostic, professionnel_de_sante et consultation. Comme ces données sont des données de santé, elles doivent être transformées dans le respect des règles de gouvernance présentées plus haut.

b. Nettoyage

Les données hospitalières étant sensibles, une sélection des champs est réalisée dès leur ingestion afin de ne conserver que les informations nécessaires aux analyses statistiques. Les données directement identifiantes (noms, prénoms, adresses, téléphones, etc.) sont supprimées lorsqu'elles ne sont pas indispensables. Les identifiants hospitaliers sont également exclus ou protégés par chiffrement et pseudo-anonymisation si leur utilisation est nécessaire.

Les données indirectement identifiantes, comme les dates de naissance ou d'hospitalisation, peuvent être supprimées ou généralisées, par exemple en utilisant des tranches d'âge. Cette sélection permet de limiter les risques de réidentification et de réduire la quantité de données sensibles stockées.

La couche Bronze contient ainsi uniquement les données utiles aux traitements futurs. Des contrôles sont également réalisés lors de l'importation pour vérifier la lisibilité des fichiers et éviter les chargements en double.

Enfin, des contrôles de qualité permettent d'identifier les valeurs manquantes, les doublons et les incohérences. Les données incorrectes sont corrigées ou rejetées selon leur importance. Le suivi des anomalies permet ainsi de garantir une meilleure fiabilité des données avant leur transformation dans les couches suivantes.

c. Transformation

1. Normalisation des données (Silver)

Après les premiers contrôles, les données sont normalisées afin de garantir une structure homogène entre les différentes sources. Les noms des colonnes sont uniformisés et les types de données corrigés si nécessaire.

Les dates sont converties dans un format commun et les valeurs numériques sont contrôlées pour détecter les erreurs ou incohérences.

Les données sont ensuite préparées pour faciliter leur rapprochement et garantir leur cohérence avant les analyses.

2. Modélisation et agrégation des données (Gold)

Le passage de Silver à Gold consiste à organiser les données selon les besoins du projet, grâce à un modèle en constellation composé de tables de faits et de dimensions.

Les dimensions décrivent les établissements, diagnostics, professionnels de santé, périodes et localisations. Les tables de faits regroupent les informations liées aux hospitalisations, consultations, mortalité et satisfaction.

Enfin, des agrégations sont réalisées pour calculer les indicateurs attendus, comme les volumes de consultations, les durées de séjour ou les taux de satisfaction, tout en respectant les règles de calcul définies.

d. Gestion des erreurs

Nous distinguons trois types d'erreurs : techniques (une brique indisponible), de format (un fichier mal formé) et de qualité (doublons, valeurs manquantes, codes inconnus). Les tâches du pipeline s'enchaînent dans un ordre strict. En cas d'échec, ClickHouse n'est pas mis à jour et les tableaux de bord conservent la dernière version valide des données.

Airflow relance automatiquement les erreurs techniques, et les lignes illisibles sont écartées dans une table de rejets. Les règles de qualité sont écrites sous forme de tests dbt, bloquants ou non selon leur gravité. Chaque échec déclenche une alerte, Grafana suit le taux de rejet et la durée des traitements, et les journaux ne contiennent aucune donnée de santé.

e. Planification des tâches

Airflow orchestre la chaîne à l'aide d'un DAG Python versionné, exécuté chaque nuit en dehors des heures d'utilisation. Les tâches s'enchaînent dans un ordre fixe : récupération du code, ingestion Spark vers Bronze, transformations et tests dbt vers Silver puis Gold, et enfin copie vers ClickHouse. Une seule exécution peut tourner à la fois. Une exécution manquée n'est pas rejouée, et un administrateur peut lancer le pipeline à la main. Airflow conserve l'historique de chaque exécution, qui alimente la supervision dans Grafana.

Plan conceptuel de données

Pour ce plan conceptuel de données, nous avons choisi de nous concentrer sur les tables de faits et de dimensions, sans intégrer les agrégations à cette première modélisation.

Nous avons opté pour un schéma en constellation afin de répondre aux besoins des utilisateurs. Les tables de faits regroupent les données sur lesquelles reposent les statistiques, tandis que les tables de dimensions permettent de filtrer et d'affiner les analyses selon différents critères.

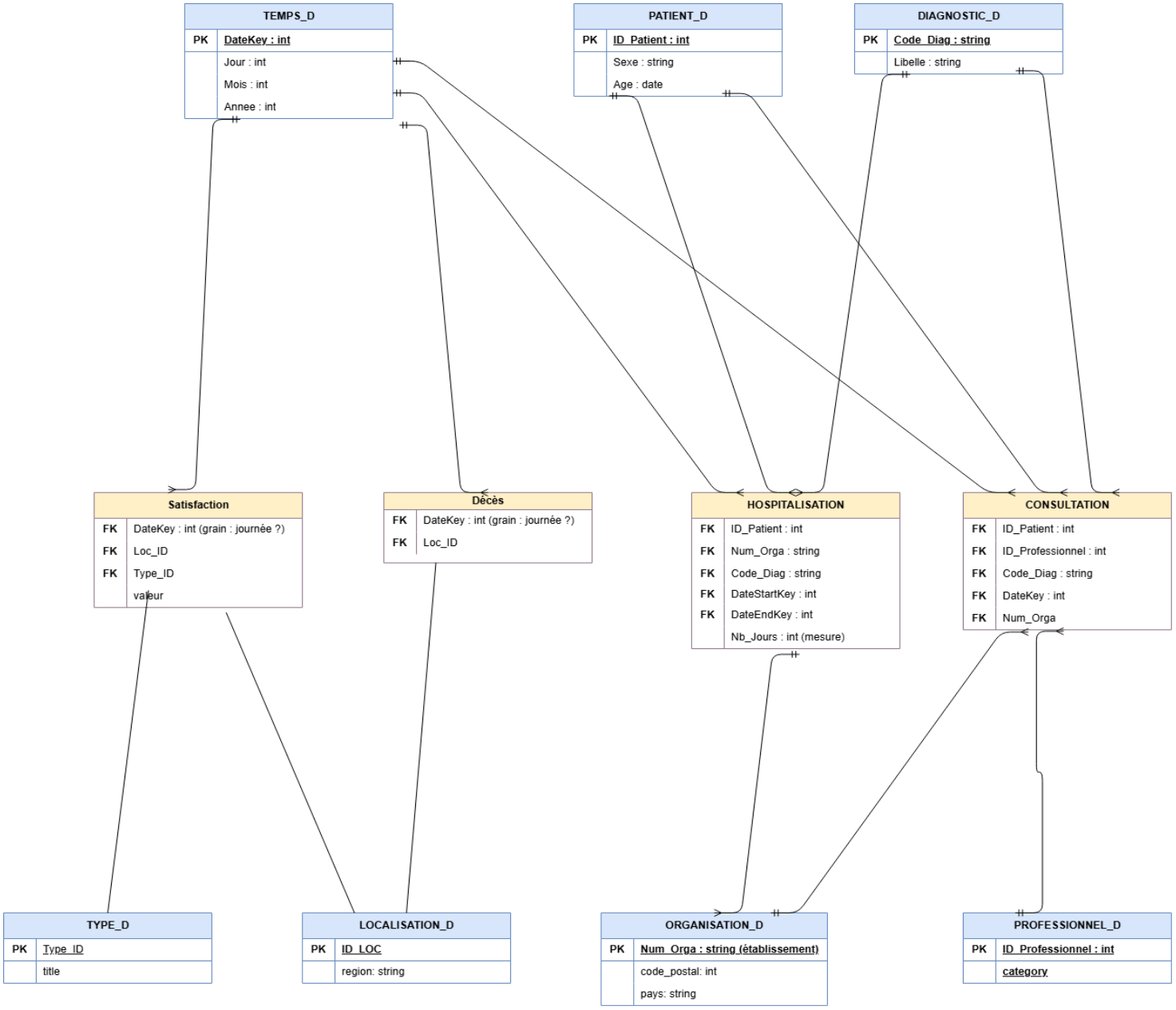


Figure 2 - Plan conceptuel de Données

a. Tables de faits

Parmi les besoins utilisateurs, nous retrouvons quatre types d'analyses : les hospitalisations, les consultations, les décès et la satisfaction. Nous avons donc créé quatre tables de faits distinctes, chacune regroupant les données nécessaires aux statistiques correspondantes.

1. Hospitalisation

Cette table regroupe les informations liées aux hospitalisations : identifiant du patient, établissement et diagnostic. Elle contient également une date de début, une date de fin et le nombre de jours d'hospitalisation. Ces éléments permettent de réaliser des analyses par période et par établissement.

2. Consultation

Cette table contient les identifiants du patient, du diagnostic, de l'établissement et du professionnel de santé, ainsi que la date de consultation. Elle permet d'analyser l'activité médicale selon différents critères.

3. Décès et satisfaction

Ces deux tables de faits sont plus simples et comportent moins de dimensions. Elles possèdent toutes deux une clé de date et une clé de localisation. La table de satisfaction contient également un identifiant du type de satisfaction et une mesure de sa valeur.

b. Table de Dimensions

Les tables de dimensions permettent de filtrer les faits pour affiner les analyses. La dimension temps, découpée en jours, est partagée par toutes les tables de faits. S'y ajoutent les dimensions diagnostic, professionnel de santé (avec la catégorie de profession), patient (sexe et âge), établissement (code postal et pays) et localisation, qui permet de filtrer par région sans passer par un établissement. Enfin, une dimension type sert à distinguer les différents scores de satisfaction.

Script d’ingestion de données

L'ingestion des données dans la couche Bronze constitue la première étape de notre pipeline de traitement. Elle consiste à récupérer les données provenant des différentes sources, notamment PostgreSQL et les fichiers CSV stockés sur S3, afin de les centraliser dans notre architecture Lakehouse. Pour cela, nous utilisons PySpark, qui permet de lire les différentes sources et d'effectuer les premiers traitements avant leur stockage au format Iceberg.

```
528     df = (
529         spark.read
530         .option("header", True)
531         .option("sep", sep)
532         .option("encoding", enc)
533         .option("multiLine", False)
534         .option("inferSchema", False)
535         .csv(f"{RAW}/{path}")
536     )
```

Figure 3 - Lecture des fichiers CSV depuis S3.

Une attention particulière est portée à la protection des données personnelles dès cette étape. Avant leur stockage, une sélection des colonnes autorisées est réalisée afin d'exclure les informations sensibles qui ne sont pas nécessaires aux analyses. Cela concerne notamment les noms, prénoms, adresses, coordonnées personnelles et certains identifiants administratifs ou hospitaliers.

```
401     excluded = EXCLUDED_COLUMNS.get(table_name, [])
402
403     allowed = [
404         column
405         for column in df.columns
406         if column not in excluded
407     ]
408
409     return df.select(*allowed)
```

Figure 4 - Exclusion des colonnes sensibles.

Lorsque certaines informations restent nécessaires au projet, elles font l'objet de transformations visant à limiter les risques de réidentification. Les identifiants des patients sont notamment pseudo-anonymisés à l'aide de l'algorithme HMAC-SHA256, associé à une clé secrète stockée dans une variable d'environnement appelée ENCRYPTION_KEY. Cette méthode permet de remplacer les identifiants d'origine par des valeurs techniques tout en conservant la possibilité de retrouver les correspondances entre les différentes tables.

D'autres transformations sont également appliquées afin de réduire la précision des informations conservées. Les dates de naissance et de décès sont généralisées par décennies, les lieux sont ramenés au niveau du département et les âges sont regroupés par

tranches. Ces opérations permettent de conserver les informations utiles aux analyses statistiques tout en limitant le niveau de détail des données personnelles.

Une fois ces traitements effectués, les données sont écrites dans des tables Iceberg au sein de la couche Bronze, stockées sur S3. Les tables provenant de PostgreSQL sont identifiées par le préfixe pg_, tandis que les fichiers CSV sont enregistrés dans des tables portant le nom de leur source.

Enfin, les noms des colonnes sont volontairement conservés tels qu'ils apparaissent dans les sources d'origine. Aucune normalisation des colonnes n'est effectuée à cette étape, car cette opération sera réalisée ultérieurement lors des transformations de la couche Silver.

À la fin de cette étape, les données provenant des différentes sources sont centralisées dans le stockage Iceberg, avec une première sélection des informations et des mesures de protection déjà appliquées. Elles constituent ainsi une base de travail pour les opérations de nettoyage, de normalisation et de fiabilisation qui seront réalisées dans la couche Silver.

Scripts de création des données

Les tables de dimensions permettent de décrire les différentes caractéristiques utilisées pour filtrer, regrouper et affiner les analyses statistiques. Elles viennent compléter les tables de faits en apportant des informations descriptives sur les patients, les établissements, les professionnels de santé, les diagnostics ou encore les périodes étudiées.

Pour leur création, nous avons utilisé le moteur MergeTree de ClickHouse, avec une clé de tri définie à partir de l'identifiant principal de chaque dimension. Certains champs utilisent également le type LowCardinality, afin d'optimiser le stockage des colonnes contenant un nombre limité de valeurs différentes.

```
CREATE TABLE dwh.dim_patient (
    `id_patient` String,
    `sexe` LowCardinality(Nullable(String)),
    `tranche_age` LowCardinality(Nullable(String))
) ENGINE = MergeTree ORDER BY id_patient SETTINGS allow_nullable_key = 1;
```

Script 1 - Création Table dim_patient

Cette dimension regroupe les informations principales concernant les patients. Elle contient leur identifiant pseudonymisé, leur sexe et leur tranche d'âge. Ces informations permettent de réaliser des analyses statistiques selon les caractéristiques des patients, tout en limitant la conservation des données personnelles.

```
CREATE TABLE dwh.dim_diagnostic (
    `code_diag` String,
    `diagnostic` Nullable(String)
) ENGINE = MergeTree ORDER BY code_diag SETTINGS allow_nullable_key = 1;
```

Script 2 - Création de la table dim_diagnostic

Cette table répertorie les différents diagnostics à partir de leur code et de leur libellé. Elle permet de retrouver facilement la description associée à chaque diagnostic et de filtrer les analyses selon les pathologies étudiées.

```
CREATE TABLE dwh.dim_professionnel (
    `id_prof_sante` String,
    `civilite` LowCardinality(Nullable(String)),
    `categorie_professionnelle` LowCardinality(Nullable(String)),
    `profession` Nullable(String),
    `specialite` Nullable(String)
) ENGINE = MergeTree ORDER BY id_prof_sante SETTINGS allow_nullable_key = 1;
```

Script 3 - Création de la table dim_professionnel

Cette dimension regroupe les informations relatives aux professionnels de santé. Elle contient leur identifiant, leur civilité, leur catégorie professionnelle, leur profession et leur

spécialité. Elle permet notamment d'analyser l'activité médicale selon les professions et les spécialités représentées.

```
CREATE TABLE dwh.dim_etablissement (
    `identifiant_organisation` String,
    `finess_site` Nullable(String),
    `raison_sociale_site` Nullable(String),
    `commune` Nullable(String),
    `code_postal` Nullable(String),
    `pays` LowCardinality(Nullable(String)),
    `departement` LowCardinality(Nullable(String))
) ENGINE = MergeTree ORDER BY identifiant_organisation SETTINGS allow_
```

Script 4 - Création de la table dim_etablissement

Cette table rassemble les informations descriptives des établissements de santé. Elle contient notamment leur identifiant d'organisation, leur numéro FINESS, leur raison sociale et leur localisation. Elle permet de comparer l'activité entre les différents établissements et d'effectuer des analyses géographiques.

```
CREATE TABLE dwh.dim_date (
    `date_jour` Date,
    `annee` Nullable(Int32),
    `trimestre` Nullable(Int32),
    `mois` Nullable(Int32),
    `jour_semaine` Nullable(Int32)
) ENGINE = MergeTree ORDER BY date_jour SETTINGS allow_nullable_key =
```

Script 5 - Création de la table dim_date

La dimension date permet de faciliter les analyses temporelles. Elle contient une date précise ainsi que plusieurs informations associées, comme l'année, le trimestre, le mois et le jour de la semaine. Cette organisation permet de filtrer et de comparer les statistiques selon différentes périodes.

```
CREATE TABLE dwh.dim_localisation (
    `code_departement` String,
    `departement` Nullable(String),
    `region` LowCardinality(Nullable(String))
) ENGINE = MergeTree ORDER BY code_departement SETTINGS allow_nullable
```

Script 6 - Création de la table dim_localisation

Cette dimension regroupe les informations géographiques à partir du code départemental. Elle permet notamment de retrouver le nom du département et sa région afin de faciliter les analyses territoriales et les comparaisons entre différentes zones géographiques.

```
CREATE TABLE dwh.dim_type_satisfaction (
    `id_type` Int32,
    `libelle` Nullable(String)
) ENGINE = MergeTree ORDER BY id_type SETTINGS allow_nullable_key = 1;
```

Script 7 - Création de la table dim_type_satisfaction

Cette dernière dimension permet de répertorier les différents critères de satisfaction étudiés dans le projet. Chaque type possède un identifiant et un libellé correspondant, comme le score global, l'accueil, la prise en charge médicale, la chambre ou encore les repas. Elle permet de distinguer les différents indicateurs présents dans la table de faits satisfaction.

Les tables de faits constituent le cœur du modèle analytique. Elles regroupent les événements et les mesures sur lesquels reposent les statistiques demandées par les utilisateurs. Chaque table correspond à un domaine d'analyse spécifique : les consultations, les hospitalisations, les décès et la satisfaction.

Elles contiennent principalement des identifiants permettant de faire le lien avec les dimensions, ainsi que des mesures nécessaires aux différents calculs statistiques. Leur structure a été définie en fonction des besoins d'analyse et des informations disponibles dans les sources.

```
CREATE TABLE dwh.fact_consultation (
    `num_consultation` Int32,
    `date_consultation` Date,
    `id_patient` Nullable(String),
    `id_prof_sante` Nullable(String),
    `code_diag` Nullable(String),
    `id_mut` Nullable(Int32),
    `duree_minutes` Nullable(Int32),
    `nb_medicaments` Nullable(Int32)
) ENGINE = MergeTree PARTITION BY toYYYYMM(date_consultation) ORDER BYdiag) SETTINGS
allow_nullable_key = 1;
```

Script 8 - Création de la table fact_consultation

Cette table regroupe les informations relatives aux consultations des patients. Elle contient le numéro de consultation, sa date, ainsi que les identifiants du patient, du professionnel de santé et du diagnostic associé. Elle comprend également plusieurs mesures, notamment la durée de consultation et le nombre de médicaments prescrits. Ces informations permettent d'étudier l'activité médicale et son évolution selon différents critères.

```
CREATE TABLE dwh.fact_hospitalisation (
    `num_hospitalisation` Int32,
    `date_entree` Date,
    `date_sortie` Nullable(Date),
    `identifiant_organisation` Nullable(String),
    `code_diag` Nullable(String),
    `duree_jours` Nullable(Int32),
    `etablissement` Nullable(String),
    `departement` LowCardinality(Nullable(String))
) ENGINE = MergeTree ORDER BY (date_entree, identifiant_organisation) SETTINGS
allow_nullable_key = 1;
```

Script 9 - Création de la table fact_hospitalisation

Cette table rassemble les informations liées aux hospitalisations. Elle contient le numéro d'hospitalisation, les dates d'entrée et de sortie, l'identifiant de l'établissement et le code du diagnostic. Elle intègre également la durée du séjour, le nom de l'établissement et son département. Elle permet ainsi d'analyser les volumes d'hospitalisation et les durées de séjour selon les établissements et les périodes.

```
CREATE TABLE dwh.fact_deces (
    `decennie_deces` Int32,
    `departement_deces` LowCardinality(Nullable(String)),
    `code_departement` LowCardinality(Nullable(String)),
    `sexe` LowCardinality(Nullable(String)),
    `decennie_naissance` Nullable(Int32),
    `age_approx` Nullable(Int32)
) ENGINE = MergeTree ORDER BY (departement_deces, decennie_deces) SETT1;
```

Script 10 - Création de la table fact_deces

Cette table regroupe les informations statistiques relatives aux décès. Elle contient notamment la décennie du décès, le département, le sexe, la décennie de naissance et l'âge approximatif. Les données sont organisées autour de caractéristiques généralisées afin de limiter les risques d'identification des personnes. Cette structure permet d'étudier la répartition des décès selon les périodes et les territoires.

```
CREATE TABLE dwh.fact_satisfaction (
    `finess` Nullable(String),
    `etablissement` Nullable(String),
    `region` LowCardinality(Nullable(String)),
    `code_departement` LowCardinality(Nullable(String)),
    `annee` Nullable(Int32),
    `id_type` Int32,
    `valeur` Nullable(Float64),
    `nb_reponses` Nullable(Int32)
) ENGINE = MergeTree ORDER BY (id_type, finess) SETTINGS allow_nullable_key = 1;
```

Script 11 - Création de la table fact_satisfaction

Cette table regroupe les résultats des enquêtes de satisfaction des établissements de santé. Elle contient l'identifiant FINESS, le nom de l'établissement, sa région, son département et l'année de l'enquête. Elle comprend également le type de satisfaction évalué, la valeur obtenue et le nombre de réponses associées. Cette organisation permet de comparer les résultats entre établissements, régions et différents critères de satisfaction.

En complément des quatre tables de faits principales, nous avons créé deux tables destinées à regrouper certains indicateurs statistiques.

```
CREATE TABLE dwh.agg_deces (
    `decennie_deces` Int32,
    `departement_deces` LowCardinality(Nullable(String)),
    `sexe` LowCardinality(Nullable(String)),
```

```
    `nb_deces` Nullable(Int32),
    `age_moyen_approx` Nullable(Float64)
) ENGINE = MergeTree ORDER BY (departement_deces, decennie_deces) SETTINGS
allow_nullable_key = 1;
```

Script 12 - Création de la table agg_deces

```
CREATE TABLE dwh.agg_satisfaction_region (
    `region` Nullable(String),
    `nb_etablissements` Nullable(Int32),
    `score_moyen` Nullable(Float64)
) ENGINE = MergeTree ORDER BY region SETTINGS allow_nullable_key = 1;
```

Script 13 - Création de la table agg_satisfaction_region

La table agg_deces permet de conserver le nombre de décès et l'âge moyen approximatif selon la décennie, le département et le sexe. La table agg_satisfaction_region regroupe quant à elle le nombre d'établissements et leur score moyen de satisfaction par région.

Ces tables permettent de disposer directement de résultats synthétiques pour faciliter certaines analyses et leur visualisation dans Superset.

Scripts de chargement des données

Après la création des différentes tables dans ClickHouse, nous avons développé des requêtes SQL avec dbt afin de les alimenter à partir des données préparées dans la couche Silver. Ces requêtes permettent de sélectionner les informations nécessaires, d'effectuer les transformations et les regroupements attendus, puis de structurer les données conformément au modèle physique défini.

Chaque requête correspond à une table spécifique et applique des traitements adaptés à son rôle dans le modèle. Certaines permettent de récupérer directement les informations, tandis que d'autres réalisent des calculs ou des agrégations afin de produire les indicateurs statistiques attendus.

Nous présentons ci-dessous trois exemples représentatifs de ces traitements. L'ensemble des autres requêtes de création et de chargement des tables est disponible en annexe 2.

```
select
    decennie_deces,
    departement_deces,
    sexe,
    cast(count(*) as integer) as nb_deces,
    avg(age_approx) as age_moyen_approx
from {{ ref('stg_deces') }}
group by 1, 2, 3
```

Script 14 - Chargement de la table agg_deces

Cette requête permet de construire une table d'agrégation regroupant les décès selon la décennie, le département et le sexe. Elle calcule le nombre total de décès à l'aide de COUNT(*), ainsi que l'âge moyen approximatif des personnes décédées avec AVG(age_approx). Ces regroupements permettent d'obtenir directement des indicateurs statistiques facilitant les analyses et les comparaisons entre territoires et périodes.

```
select
    id_patient,
    sexe,
    age as tranche_age  -- tranche déjà généralisée à l'ingestion (ex.
'41-50 ans')
from {{ ref('stg_patient') }}
```

Script 15 - Chargement de la table dim_patient

Cette requête permet d'alimenter la dimension patient à partir des données préparées dans la couche Silver. Elle récupère l'identifiant pseudonymisé du patient, son sexe et sa tranche d'âge. Cette dernière ayant déjà été généralisée lors de l'ingestion, elle

permet de conserver une information utile aux analyses tout en limitant la précision des données personnelles.

```
{{ config(properties={"partitioning": "ARRAY['decennie_deces']"}) }}
select
    decennie_deces,
    departement_deces,
    code_departement,
    sexe,
    decennie_naissance,
    age_approx
from {{ ref('stg_deces') }}
```

Script 16 - Chargement de la table fact_deces

Cette requête permet de récupérer les informations nécessaires à l'analyse des décès depuis la table intermédiaire correspondante. Elle conserve notamment la décennie du décès, le département, le sexe, la décennie de naissance et l'âge approximatif. Une configuration de partitionnement par décennie de décès est également définie afin d'organiser les données et de faciliter les requêtes portant sur différentes périodes.

Partitionnement et indexation

Une table MergeTree stocke ses données en parts immuables, triées selon la clé ORDER BY. Trois mécanismes limitent les données lues (figure 3) :

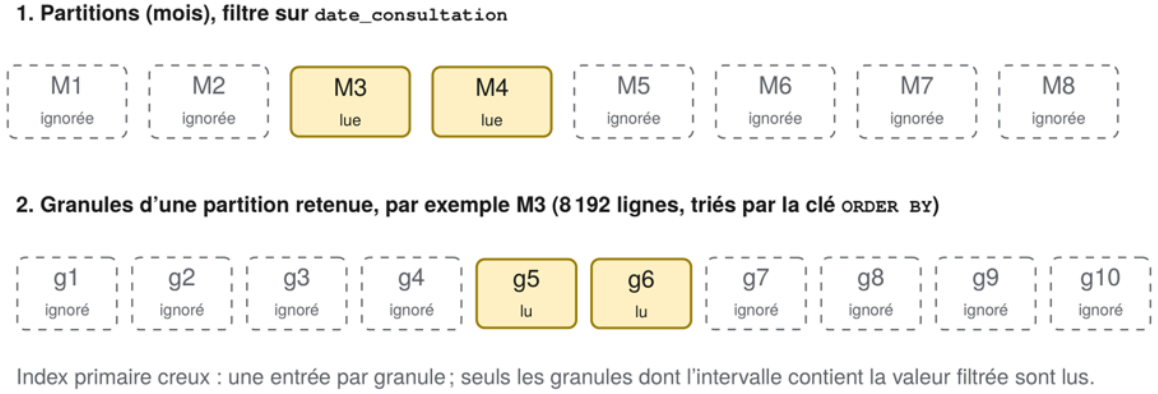


Figure 5 - Double élagage : partitions puis granules (schéma illustratif)

Le partitionnement (PARTITION BY) range les parts par valeur, par exemple le mois : un filtre sur la date n'ouvre que les partitions concernées. Il sert d'abord à gérer les données (purge d'un mois), et chaque partition ajoute au moins une part; l'index primaire creux, construit sur la clé de tri, garde une entrée par granule de 8 192 lignes : un filtre sur les premières colonnes de la clé écarte les granules qui ne contiennent pas la valeur cherchée; les index de saut (minmax, set, bloom_filter) résument des groupes de granules pour des colonnes hors de la clé de tri.

a. Stratégie par table

Dans Iceberg (Gold), les faits sont partitionnés par la transformation month de leur date, et par décennie pour les décès; les petites tables ne le sont pas. Dans ClickHouse, la clé de tri place d'abord les colonnes filtrées par les requêtes métier, puis la date (tableau 10). Les index de saut couvrent des filtres hors clé de tri : bloom_filter sur id_patient, minmax sur les durées et la décennie de naissance, set sur le département des hospitalisations, avec une entrée pour 4 granules.

Table	PARTITION BY	ORDER BY	Justification
`fact_hospitalisation`	`toYYYYMM(date_entree)`	`(identifiant_organisation, code_diag, date_entree)`	Requêtes par établissement, diagnostic et période ; le mois permet d'ignorer les périodes hors filtre.
`fact_consultation`	`toYYYYMM(date_consultation)`	`(code_diag, id_prof_sante, date_consultation)`	Requêtes par diagnostic et par professionnel sur une période.

fact_deces	decennie_deces	(departement_deces, decennie_deces)	Indicateurs par département et par décennie ; table la plus volumineuse (environ 25 millions de lignes), partition par décennie car les dates sont généralisées.
fact_satisfaction	aucune	(finess)	Petite table : partitionner créerait des parts minuscules sans gain.
dimensions	aucune	clé primaire	Jusqu'à un million de lignes (professionnels) ; lues en entier par les jointures, la partition n'aiderait pas.

Tableau 1 - Partitionnement et clé de tri proposés par table

Table	Lignes	Taille sur disque (Mo, A0)	Partitions (A1)
fact_hospitalisation	2 479	0,1	73
fact_consultation	1 027 157	86,1	94
fact_deces	25 087 507	66,0	13
fact_satisfaction	1 152	0,04	1

Tableau 2 - Volumétrie des tables de l'entrepôt

Remarque. Cette stratégie est l'hypothèse de départ du benchmark. Les mesures confirment la clé de tri de fact_deces, mais montrent que le partitionnement mensuel ne fait pas gagner de temps sur fact_consultation et ralentit fact_hospitalisation : 73 partitions pour 2 479 lignes (section 7.5).

b. Scripts

Le script 1 crée la configuration A2 (partition et clé de tri) et le script 2 ajoute les index de saut (A3). La configuration A1 et le partitionnement Iceberg (20_iceberg_partitioning.sql) suivent le même modèle et sont dans l'archive.

```
29  -- ------------------------------------------------------------------ A2
 30  -- Clé de tri : colonnes les plus filtrées d'abord (haute sélectivité), date en dernier.
 31  CREATE TABLE dwh_a2.fact_consultation AS dwh_a0.fact_consultation
 32    ENGINE = MergeTree PARTITION BY toYYYYMM(date_consultation)
 33    ORDER BY (code_diag, id_prof_sante, date_consultation);
 34  CREATE TABLE dwh_a2.fact_hospitalisation AS dwh_a0.fact_hospitalisation
 35    ENGINE = MergeTree PARTITION BY toYYYYMM(date_entree)
 36    ORDER BY (identifiant_organisation, code_diag, date_entree);
 37  CREATE TABLE dwh_a2.fact_deces AS dwh_a0.fact_deces
 38    ENGINE = MergeTree PARTITION BY decennie_deces
 39    ORDER BY (departement_deces, decennie_deces);
 40  CREATE TABLE dwh_a2.fact_satisfaction AS dwh_a0.fact_satisfaction
 41    ENGINE = MergeTree ORDER BY (finess);
```

Script 17 - Partition et clé de tri des tables de faits (A2) (extrait, lignes 29 à 41)

```
21    ENGINE = MergeTree PARTITION BY toYYYYMM(date_consultation)
 22    ORDER BY (code_diag, id_prof_sante, date_consultation);
 23    ALTER TABLE dwh_a3.fact_consultation ADD INDEX idx_patient id_patient TYPE bloom_filter(0.01)
GRANULARITY 4;
 24  ALTER TABLE dwh_a3.fact_consultation ADD INDEX idx_duree duree_minutes TYPE minmax GRANULARITY 4;
 25
 26  CREATE TABLE dwh_a3.fact_hospitalisation AS dwh_a2.fact_hospitalisation
 27    ENGINE = MergeTree PARTITION BY toYYYYMM(date_entree)
 28    ORDER BY (identifiant_organisation, code_diag, date_entree);
 29  ALTER TABLE dwh_a3.fact_hospitalisation ADD INDEX idx_jours duree_jours TYPE minmax GRANULARITY 4;
 30  ALTER TABLE dwh_a3.fact_hospitalisation ADD INDEX idx_dept departement TYPE set(200) GRANULARITY 4;
 31
 32  CREATE TABLE dwh_a3.fact_deces AS dwh_a2.fact_deces
 33    ENGINE = MergeTree PARTITION BY decennie_deces
 34    ORDER BY (departement_deces, decennie_deces);
 35  ALTER TABLE dwh_a3.fact_deces ADD INDEX idx_naiss decennie_naissance TYPE minmax GRANULARITY 4;
 36
```

Script 18 - Index de saut (A3) (extrait, lignes 21 à 36)

Rapport de Qualité des données

La qualité est mesurée sur la couche Bronze (avant nettoyage) et sur la couche Silver (après), par les requêtes de sql/quality/ exécutées sur Trino. Trois taux sont calculés : la complétude d'un attribut, $(N - N_{manq}) / N$, où une valeur manquante est nulle, vide ou conventionnelle (NA, 0000-00-00) ; le taux d'orphelins d'une clé étrangère, part des lignes dont la référence n'existe pas dans la table parente ; le taux de doublons d'une clé, $(N - N_{dist}) / N$.

a. Complétude

Le tableau 19 de l'annexe 2 donne la complétude de chaque attribut de la couche Silver et, lorsque la colonne existe sous le même nom, celle de la couche Bronze. Les tables métier issues de PostgreSQL (patient, consultation, professionnel) et l'hospitalisation sont complètes à 100 % : aucune valeur nulle, vide ou conventionnelle. Les écarts se concentrent sur trois sources : l'annuaire des établissements (finess_site renseigné pour 200 lignes sur 416 665, soit 0,05 % ; commune à 66,7 %), la satisfaction (74,5 % de scores renseignés, les autres établissements n'ayant pas assez de réponses) et les décès (99,6 % de décennies de naissance exploitables).

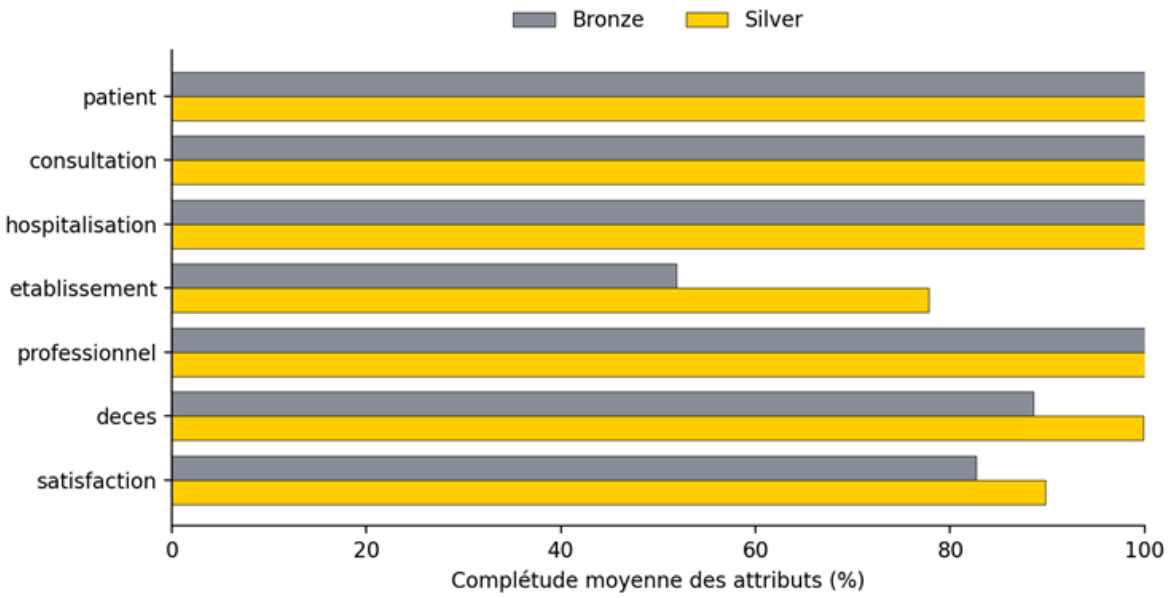


Figure 6 - Taux de complétude par table, avant et après nettoyage

b. Cohérence référentielle et codes FINESS

Deux clés relient les établissements : l'identifiant d'organisation (hospitalisations et annuaire) et le code FINESS, 9 caractères dont les deux premiers donnent le département (satisfaction et annuaire). Le contrôle vérifie la présence de chaque référence dans la table parente et le format des codes FINESS.

Référence (enfant)	Cible (parent)	Taux d'orphelins	Traitement
hospitalisations.identifiant_organisation	etablissement_sante.identifiant_organisation	0 % (0 sur 2 479)	Aucun traitement nécessaire ; la valeur « INCONNU » reste prévue si une clé manque.
satisfaction.finess	etablissement_sante.finess_site	100 % (1 152 sur 1 152)	Lignes conservées ; rapprochement avec l'annuaire impossible, signalé (voir synthèse).
consultation.id_patient	patient.id_patient	0 % (0 sur 1 027 157)	Aucun traitement nécessaire.
consultation.id_prof_sante	professionnel.identifiant	0 % (0 sur 1 027 157)	Aucun traitement nécessaire.
consultation.code_diag	diagnostic.code_diag	0 % (0 sur 1 027 157)	Aucun traitement nécessaire.
hospitalisations.code_diagnostic	diagnostic.code_diag	0 % (0 sur 2 479)	Aucun traitement nécessaire.
Codes FINESS au format invalide (hors du format attendu)		0 % (0 sur 200 en établissement, 0 sur 1 152 en satisfaction)	Aucun code à exclure.

Tableau 3 - Règles de cohérence référentielle et résultats

Toutes les références des consultations et des hospitalisations sont valides. En revanche, aucun des 1 152 codes FINESS de la satisfaction ne figure dans l'annuaire : celui-ci ne renseigne le FINESS que pour 200 de ses 416 665 lignes. Le lien satisfaction–établissement est donc impossible avec les sources actuelles ; fact_satisfaction reste exploitable par région. Action : signaler la limite et obtenir un référentiel FINESS complet.

c. Doublons

Entité	Clé métier	Taux de doublons	Traitement
Patient	id_patient	0 % (100 000 lignes, 100 000 clés)	Unicité testée par dbt ; aucun traitement.
Consultation	num_consultation	0 % (1 027 157 lignes)	Unicité testée par dbt ; aucun traitement.
Hospitalisation	num_hospitalisation	0 % (2 479 lignes)	Unicité testée par dbt ; aucun traitement.

Établissement	identifiant_organisation	0 % (416 665 lignes en Bronze)	Une ligne par organisation déjà garantie (row_number() dans stg_etablissement) ; aucune ligne supprimée.
Décès	ensemble des colonnes	non significatif (25 087 550 lignes pour 15 904 combinaisons)	Aucune suppression : voir ci-dessous.

Tableau 4 - Doublons par entité et traitement

Pour les décès, 25 087 550 lignes ne forment que 15 904 combinaisons distinctes : l'anonymisation ramène chaque décès à une décennie, un département et un sexe, si bien que des personnes différentes deviennent identiques. Ce ne sont pas des doublons ; les supprimer ferait perdre 99,9 % des décès.

d. Contrôles automatiques

Les tests dbt rejouent ces règles à chaque chargement : unique et not_null sur les clés (bloquants), relationships sur les clés étrangères et un test de format FINESS (avertissements, pour ne pas bloquer le chargement quotidien sur une anomalie de source). Le contrôle des volumes a aussi révélé que fact_deces était vide (tableau 9).

Logs et traces

a. Logs, métriques et traces

Un log est une ligne de texte horodatée qui décrit un événement dans une brique : une erreur, une requête exécutée, une tâche terminée.

Une métrique est une mesure chiffrée suivie dans le temps, comme la mémoire utilisée ou la durée d'une tâche.

Une trace suit une même opération à travers plusieurs composants : elle est découpée en étapes reliées par un identifiant commun, chacune avec sa durée, ce qui permet de voir où le temps est perdu.

Dans ce projet, « traces » désigne aussi la traçabilité au sens du RGPD et du référentiel HDS, c'est-à-dire la possibilité de dire qui a accédé à quelles données et quelle exécution a produit quel résultat.

b. Centralisation des logs

Chaque brique de la chaîne écrit ses propres journaux. Sans centralisation, chercher la cause d'un incident oblige à ouvrir les outils un par un. Nous avons retenu Grafana Loki, alimenté par Grafana Alloy et consulté dans Grafana. Loki n'indexe que des étiquettes (service, tâche, niveau) et stocke le texte compressé, ce qui le rend plus léger qu'Elasticsearch. La recherche dans le texte reste possible avec le langage de requête LogQL.

Alloy lit les sorties standard de tous les conteneurs Docker, y compris ceux de Spark lancés par Airflow, ainsi que les fichiers de logs des tâches Airflow, écrits en JSON. Chaque ligne reçoit les étiquettes service, container et job, et les tâches Airflow ajoutent dag_id, task_id et level. Deux requêtes suffisent à illustrer l'usage : `{service="trino"} |= "ERROR"` retrouve les erreurs de Trino, et `{job="airflow-task", task_id="dbt_build", level="error"}` isole celles de l'étape dbt. Le tableau de bord « Logs lakehouse » affiche le nombre d'erreurs par service et une recherche libre. Ces logs sont conservés 30 jours.

c. Métriques et supervision

Alloy collecte aussi les métriques et les envoie à Prometheus, avec la même rétention de 30 jours. Elles couvrent la machine hôte, chaque conteneur, PostgreSQL, Airflow (via StatsD) et les services qui exposent leurs propres métriques : Trino, ClickHouse, SeaweedFS, Nessie, Loki et Grafana. Un tableau de bord d'exploitation les regroupe. Il aide à repérer des anomalies qui ne provoquent pas d'échec, comme un traitement

anormalement long, avant que les tableaux de bord métier n'affichent des données périmées.

d. Traçabilité des accès aux données

Trino est le point d'accès aux données pour dbt et Superset. Chaque fois qu'une requête se termine, son http-event-listener la signale à un petit service que nous avons écrit, trino-audit, qui la reformate en une ligne JSON sur sa sortie standard. Cette ligne contient l'utilisateur, l'adresse IP du client, l'application source, les tables et colonnes lues, l'état de la requête, le nombre de lignes, son identifiant et son texte. Alloy la collecte comme n'importe quel autre log et le tableau de bord « Audit accès données » permet de la consulter par utilisateur ou par table. Pour savoir qui a lu la dimension patient, par exemple, la requête `{service="trino-audit"} | json | tables=~".*dim_patient.*"` suffit.

Ces logs d'audit sont conservés un an, grâce à une règle de rétention propre au flux trino-audit, alors que les logs techniques ne le sont que 30 jours. Les deux durées restent à confirmer avec la politique de conservation du plan de gouvernance.

e. Traçabilité des exécutions

Airflow garde l'historique de chaque exécution du pipeline, identifiée par dag_id, run_id et task_id, avec les logs de chaque tentative. On peut ainsi savoir quelle tâche a produit quelles données et pourquoi elle a échoué. Les identifiants de requête de Trino permettent de relier une ligne d'audit aux logs du service qui l'a exécutée.

f. Données de santé dans les logs

Les journaux ne doivent pas devenir une copie non maîtrisée des données sensibles. La pseudo-anonymisation par HMAC-SHA256 et la généralisation des dates et des âges sont appliquées dès l'ingestion, donc les logs ne contiennent que des identifiants techniques et des valeurs déjà généralisées.

Benchmark et performances

Cette partie mesure l'effet du partitionnement, de la clé de tri et des index de saut sur les temps de réponse de ClickHouse. Les mêmes données sont chargées dans quatre configurations, interrogées avec huit requêtes issues des besoins des praticiens et de la direction.

a. Configurations comparées

Les mêmes données sont chargées dans quatre bases ClickHouse. Chaque configuration ajoute une seule optimisation à la précédente, ce qui attribue chaque écart à sa cause :

- A0 (dwh_a0) : types et contraintes, aucune optimisation (ORDER BY tuple()) ;
- A1 (dwh_a1) : A0 plus le partitionnement des tables de faits, par mois, et par décennie pour les décès ;
- A2 (dwh_a2) : A1 plus une clé de tri propre à chaque table de faits ;
- A3 (dwh_a3) : A2 plus des index de saut.

Les dimensions, identiques partout, sont lues dans dwh_a0. La base de production dwh, lue par Superset, n'est pas modifiée par le benchmark.

b. Protocole

Chaque requête est exécutée 5 fois par configuration après une exécution de chauffe, à froid (caches de ClickHouse vidés avant chaque exécution) et à chaud ; le cache de requêtes est désactivé. Durée, lignes lues et octets lus viennent de system.query_log. Le nombre de lignes et une somme de contrôle du résultat sont comparés entre configurations, pour vérifier que les optimisations ne changent pas les réponses.
Machine : 4 cœurs Xeon Gold 6248, 15,6 Gio de mémoire, SSD, ClickHouse 26.9, sans pipeline en cours mais partagée avec les autres services du Lakehouse.

c. Requêtes de benchmark

Les requêtes reprennent les analyses attendues par les utilisateurs : taux de consultation et d'hospitalisation selon l'établissement, le diagnostic, le professionnel, le sexe, l'âge ou la période, puis mortalité par département et satisfaction par région. Le fichier d'hospitalisation ne contient pas d'identifiant patient : le croisement par sexe et par âge porte donc sur les consultations. Huit requêtes en couvrent les variantes, dont une requête témoin sans filtre.

Id	Indicateur métier	Tables	Levier attendu
Q1	Consultations par mois et par diagnostic sur une année	fact_consultation, dim_diagnostic	Élagage des partitions (filtre sur la date)
Q2	Consultations par catégorie professionnelle et par trimestre sur deux ans	fact_consultation, dim_professionnel, dim_date	Élagage des partitions ; coût de la jointure
Q3	Durée moyenne de séjour par mois pour un établissement donné	fact_hospitalisation	Clé de tri (préfixe identifiant_organisation)
Q4	Consultations par tranche d'âge et par sexe sur une année	fact_consultation, dim_patient	Élagage des partitions ; jointure sur id_patient
Q5	Décès par décennie pour un département donné	fact_deces	Clé de tri (departement_deces) et partition par décennie
Q6	Score de satisfaction moyen par région	fact_satisfaction	Aucun : petite table, requête de contrôle
Q7	Dix diagnostics les plus fréquents d'un établissement sur un trimestre	fact_hospitalisation, dim_diagnostic	Clé de tri et élagage des partitions combinés
Q8	Comptage et moyenne sans filtre sur une table de faits (témoin)	fact_deces	Aucun : lecture complète

Tableau 5 - Requêtes de benchmark et levier d'optimisation attendu

Toutes les requêtes sont dans sql/benchmark/queries.sql, paramétrées (période, établissement, département). Exemple, requête Q5 :

```
SELECT decennie_deces, count() AS nb_deces, avg(age_approx) AS
age_moyen_approx
FROM fact_deces
WHERE departement_deces = {dept:String}
GROUP BY decennie_deces
ORDER BY decennie_deces;
```

Script 19 - SQL Requête Q5

Le script run_benchmark.py exécute les requêtes et run_all.sh enchaîne chargement, configurations, mesures et rapport de qualité.

d. Résultats avant et après optimisation

Le tableau 2 donne, pour chaque requête, le temps à chaud dans chaque configuration, le gain de A3 sur A0 (temps A0 divisé par temps A3 : au-dessus de 1, A3 est plus rapide) et les lignes lues, qui montrent directement l'effet de l'élagage. Le fichier benchmark/resultats.csv contient chaque exécution, à froid et à chaud.

Id	A0 (ms)	A1 (ms)	A2 (ms)	A3 (ms)	Gain A3/A0	Lignes lues A0	Lignes lues A3
Q1	1460	1570	1630	1520	×0,96	1.04 M	1.04 M
Q2	990	1150	1370	1180	×0,84	2.08 M	2.08 M
Q3	30	20	20	30	×1,00	2.48 K	37
Q4	1008	1190	1210	1410	×0,71	1.13 M	1.13 M
Q5	500	420	60	50	×10	18.03 M	344.07 K
Q6	20	10	20	20	×1	1.15 K	1.15K
Q7	400	130	220	200	×2	17.97 K	17.95 K
Q8	230	200	240	250	×0,92	25.09 M	25.09 M

Tableau 6 - Temps à chaud (ms)

Les lignes lues comprennent celles de la dimension jointe (par exemple 15 490 lignes de dim_diagnostic pour Q1 et Q7).

Mesure	A0	A1	A2	A3
Taille sur disque des quatre faits (Mo)	152,3	154,3	150,6	153,2
Durée de chargement des faits (s)	22,3	21,1	29,8	23,5
Nombre de parts	11	191	191	201

Tableau 7 - Coût des optimisations : taille sur disque et durée de chargement

e. Analyse et interprétation des résultats

Avant les mesures, chaque optimisation a été associée à un effet attendu, déduit du fonctionnement de MergeTree. Le tableau 4 confronte ces hypothèses aux résultats.

Hypothèse	Constat	Verdict
Gain de A0 à A1 sur les requêtes filtrées par date (Q1, Q2, Q4) : élagage des parts hors période	Mêmes volumes de lignes lues (ex. 1.04 M pour Q1, 2.08 M pour Q2, 1.13 M pour Q4) et temps légèrement augmentés ou équivalents en A3	Infirmée
Gain de A1 à A2 sur les requêtes filtrées par établissement ou département (Q3, Q5, Q7) : l'index primaire écarte les granules inutiles	Q5 : 18.03 M à 344.07 K lignes lues, temps divisé par 10 (500 ms à 50 ms). Q7 : passage de 400 ms à 200 ms (x2). Q3 : réduction drastique des lignes lues (2.48 K à 37)	Confirmée pour Q5 (très forte sur Q5, visible sur Q7)
Gain faible ou nul de A2 à A3 : les filtres sont déjà couverts par la clé de tri	Aucun gain significatif (temps A2 et A3 très proches)	Confirmée
Aucun gain sur Q6 et Q8 : petite table ou lecture complète	Lignes lues strictement identiques (1.15 K pour Q6, 25.09 M pour Q8) ; variations négligeables dues au bruit	Confirmée
Dégradation possible : trop de petites parts	Sur Q1, Q2 et Q4, le découpage en parts n'apporte aucun élagage et engendre un léger surcoût de gestion (ex. Q4 passe de 1008 ms à 1410 ms)	Observée

Tableau 8 - Hypothèses de départ et constats mesurés

La clé de tri est la seule optimisation rentable.

Q5 filtre sur le département, première colonne de la clé de tri de fact_deces. En A3, les lignes de chaque département sont contiguës : l'index primaire ne lit que les granules du département cherché, soit 344,07 milliers de lignes au lieu de 18,03 millions en A0. Cela se traduit par un gain d'un facteur 10 sur le temps d'exécution (50 ms contre 500 ms).

Lire moins de lignes ne suffit pas à gagner du temps.

Sur fact_hospitalisation (Q3 et Q7), l'élagage réduit fortement le volume de données lues (de 2.48 K à 37 lignes pour Q3, et de 17.97 K à 17.95 K pour Q7, permettant à Q7 de passer de 400 ms à 200 ms). En revanche, sur les consultations (Q1, Q2, Q4), l'absence d'élagage sur les lignes lues (1.04 M, 2.08 M et 1.13 M) combinée au surcoût de gestion des multiples parts fait passer Q4 de 1008 ms (A0) à 1410 ms (A3).

Partitionner une petite table la ralentit.

fact_hospitalisation compte seulement 2 479 lignes. Son partitionnement la découpe en de multiples petites parts d'une trentaine de lignes : cela ajoute un coût d'ouverture, de balayage et de fusion de parts qui dégrade les performances sur les petites tables. Même si l'élagage réduit le nombre de lignes lues (Q3 passe de 2,48 K à 37 lignes), le temps global ne gagne rien (30 ms) ou se dégrade par rapport aux exécutions légères. Le partitionnement doit donc être réservé aux tables volumineuses.

Froid, chaud et coût.

Les temps à froid sont proches des temps à chaud : les tables sont de taille modérée et le cache du système reste très actif. L'espace disque varie de moins de 3 % entre les configurations ; le vrai coût réside dans la multiplication des parts, qui passe de 11 en A0 à 201 en A3 (tableau 3).

Écarts attribués au bruit de la machine.

Les petites variations observées sur des requêtes quasi-instantanées comme Q3 et Q6 (entre 10 ms et 30 ms) ou sur Q8 (230 ms en A0 contre 250 ms en A3 pour un volume identique de 25,09 M de lignes) relèvent de la marge d'erreur et de la charge momentanée de la machine partagée.

Stratégie corrigée.

Les mesures conduisent à réviser la stratégie de la partie Partitionnement et indexation :

- conserver la clé de tri sur le département pour fact_deces ;
- ne plus partitionner fact_hospitalisation ni fact_satisfaction ;
- garder la partition mensuelle de fact_consultation pour la seule gestion des données (purge d'un mois, durée de conservation), sans en attendre de gain de temps à ce volume ;
- retirer les index de saut tant qu'aucune requête ne les exploite.

Les huit requêtes renvoient des résultats identiques dans les quatre configurations (même nombre de lignes et même somme de contrôle) : les optimisations ne modifient pas les réponses.

f. Limites de la mesure

Les essais portent sur une machine unique et sur le volume de données du projet : les gains observés ne se transposent pas tels quels à un volume très supérieur ou à plusieurs serveurs.

Les temps dépendent de l'état des caches. La distinction entre mesure à froid et mesure à chaud limite ce biais sans l'éliminer, car le cache du système de fichiers de la machine reste actif.

Le benchmark exécute les requêtes une par une : il ne mesure pas le comportement sous charge concurrente, par exemple plusieurs tableaux de bord Superset ouverts en même temps.

Conclusion

Le projet Cloud Healthcare Unit vise à créer une plateforme décisionnelle permettant de centraliser et d'analyser les données hospitalières à des fins statistiques. L'architecture repose sur un modèle Lakehouse organisé en trois couches : Bronze pour le stockage initial, Silver pour le nettoyage et la fiabilisation, et Gold pour la production des indicateurs. Les différentes technologies retenues assurent le traitement, le stockage, l'orchestration et la visualisation des données. La modélisation en constellation permet de répondre aux besoins des utilisateurs grâce aux tables de faits et de dimensions.

Sécurité et qualité des données

La qualité des données repose sur le nettoyage des formats, des dates, des doublons et des valeurs manquantes. Les informations directement identifiantes non nécessaires sont supprimées dès l'ingestion. Des mesures de pseudo-anonymisation, de chiffrement et de contrôle des accès sont également prévues afin de respecter le RGPD et les exigences liées aux données de santé.

Performances

Le benchmark a permis de confronter nos choix d'optimisation à des mesures réelles. La clé de tri est la seule optimisation vraiment rentable : elle divise par dix le temps de réponse sur les décès. À l'inverse, le partitionnement mensuel n'apporte aucun gain à ce volume et ralentit les petites tables. Ces résultats nous ont conduits à revoir notre stratégie. Nous conservons les clés de tri, nous supprimons le partitionnement des petites tables et nous retirons les index de saut inutilisés.

Perspectives

Plusieurs pistes restent à approfondir. Il faudra obtenir un référentiel FINESS complet pour relier la satisfaction aux établissements, confirmer les durées de conservation des logs avec le plan de gouvernance, et mesurer le comportement de la plateforme sous charge concurrente. L'intégration de Pro Santé Connect permettra enfin de simplifier l'accès des professionnels de santé.

L'architecture est définie, le modèle de données est construit et alimenté, la gouvernance est cadrée et les performances ont été mesurées puis optimisées. La chaîne complète fonctionne de l'ingestion jusqu'à ClickHouse. Il reste à exploiter les indicateurs produits dans Superset pour répondre concrètement aux besoins des praticiens et de la direction.

Annexes

Annexe 1 : Glossaire

AIPD	Analyse d'Impact relative à la Protection des Données
AM	Assurance Maladie
ANS	Agence du Numérique en Santé
DAG	Directed Acyclic Graph
DBT	DataBase Tool
CHU	Cloud Healthcare Unit
CNIL	Commission Nationale de l'Informatique et des Libertés
CPE	Carte de Professionnel d'Établissement
CPF	Carte de Professionnel en Formation
CPS	Carte de Professionnel de Santé
CPx	Carte de Professionnel de tous types
ELT	Extract, Load, Transform
ETL	Extract, Transform, Load
FINESS	Fichier National des Établissements Sanitaires et Sociaux
FTP	File Transfert Protocol
HCG	Health Care Group
HDFS	Hadoop Distributed File System
HDS	Hébergeur de Données de Santé
INSEE	Institut National de la Statistique et des Etudes Economiques
JDBC	Java Database Connectivity
LDAP	Lightweight Directory Access Protocol
MFA	Multi-Factor Authentication
PSC	Pro Santé Connect
OIDC	OpenId Connect

PGSSI-S	Politique Générale de Sécurité des Systèmes d'Information de Santé
RGPD	Règlement Général de la Protection des Données
SGBD	Système de Gestion de Base de Donnée
SSD	Solid-State Drive
SSO	Single Sign-On
UE	Union Européenne

Annexe 2 : Requêtes de création des tables et de chargement des données

```
select
    region,
    cast(count(*) as integer) as nb_etablissements,
    avg(score_global) as score_moyen
from {{ ref('stg_satisfaction') }}
where score_global is not null
group by 1
```

```
with dates as (
    select date_consultation as d from {{ ref('stg_consultation') }}
    union
    select date_entree from {{ ref('stg_hospitalisation') }}
    union
    select date_sortie from {{ ref('stg_hospitalisation') }}
)
select
    d as date_jour,
    cast(year(d) as integer) as annee,
    cast(quarter(d) as integer) as trimestre,
    cast(month(d) as integer) as mois,
    cast(day_of_week(d) as integer) as jour_semaine
from dates
where d is not null
```

```
select code_diag, max(diagnostic) as diagnostic from {{
ref('stg_diagnostic') }}
group by code_diag
```

```
select * from {{ ref('stg_etablissement') }}
```

```
select code_departement, departement, region from {{
ref('stg_localisation') }}
```

```
select * from {{ ref('stg_professionnel') }}
```

```
select id_type, libelle
from (values
    (1, 'Score global'),
    (2, 'Accueil'),
    (3, 'Prise en charge par les infirmiers'),
    (4, 'Prise en charge par les médecins'),
    (5, 'Chambre'),
    (6, 'Repas'),
    (7, 'Sortie')
) as t (id_type, libelle)
```

```
{{ config(properties={"partitioning":
"ARRAY['month(date_consultation)']"}) }}
with prescriptions as (
    select num_consultation, cast(count(*) as integer) as nb_medicaments
    from {{ source('bronze', 'pg_prescription') }}
    group by 1
)
```

```
select
    c.num_consultation,
    c.date_consultation,
    c.id_patient,
    c.id_prof_sante,
    c.code_diag,
    c.id_mut,
    c.duree_minutes,
    coalesce(p.nb_medicaments, 0) as nb_medicaments
from {{ ref('stg_consultation') }} c
left join prescriptions p on p.num_consultation = c.num_consultation
```

```
{{ config(properties={"partitioning": "ARRAY['month(date_entree)']"}) }}
select
    h.num_hospitalisation,
```

```
    h.date_entree,
    h.date_sortie,
    h.identifiant_organisation,
    h.code_diag,
    h.duree_jours,
    e.raison_sociale_site as etablissement,
    e.departement
from {{ ref('stg_hospitalisation') }} h
left join {{ ref('stg_etablissement') }} e on e.identifiant_organisation
= h.identifiant_organisation
```

```
with scores as (
    select finess, etablissement, region, annee, 1 as id_type,
score_global as valeur, nb_reponses as nb_reponses
    from {{ ref('stg_satisfaction') }}
    where score_global is not null
    union all
    select finess, etablissement, region, annee, 2 as id_type,
score_accueil as valeur, nb_rep_accueil as nb_reponses
    from {{ ref('stg_satisfaction') }}
    where score_accueil is not null
    union all
    select finess, etablissement, region, annee, 3 as id_type,
score_pec_infirmier as valeur, nb_rep_pec_infirmier as nb_reponses
    from {{ ref('stg_satisfaction') }}
    where score_pec_infirmier is not null
    union all
    select finess, etablissement, region, annee, 4 as id_type,
score_pec_medical as valeur, nb_rep_pec_medical as nb_reponses
    from {{ ref('stg_satisfaction') }}
    where score_pec_medical is not null
    union all
    select finess, etablissement, region, annee, 5 as id_type,
score_chambre as valeur, nb_rep_chambre as nb_reponses
    from {{ ref('stg_satisfaction') }}
    where score_chambre is not null
    union all
    select finess, etablissement, region, annee, 6 as id_type,
score_repas as valeur, nb_rep_repas as nb_reponses
    from {{ ref('stg_satisfaction') }}
    where score_repas is not null
    union all
    select finess, etablissement, region, annee, 7 as id_type,
score_sortie as valeur, nb_rep_sortie as nb_reponses
    from {{ ref('stg_satisfaction') }}
```

```
    where score_sortie is not null
)
select
    s.finess,
    s.etablissement,
    s.region,
    l.code_departement,
    s.annee,
    cast(s.id_type as integer) as id_type,
    s.valeur,
    s.nb_reponses
from scores s
left join {{ ref('stg_etablissement') }} e on e.finess_site = s.finess
left join {{ ref('stg_localisation') }} l on l.code_departement =
e.departement
```